

Imbalance Datasets

An imbalanced dataset occurs when the distribution of classes or labels in the dataset is highly skewed, meaning that one or more classes are significantly more prevalent than others. This can present challenges in various domains, including healthcare, finance, and cybersecurity. For instance, in medical diagnosis, the occurrence of a rare disease may be far less frequent than that of common ailments, resulting in an imbalanced dataset where positive cases are heavily outnumbered by negative cases. In financial fraud detection, fraudulent transactions may represent only a small fraction of overall transactions, leading to an imbalanced dataset where the majority of instances are non-fraudulent. Addressing class imbalance is crucial for building effective machine learning models that can accurately classify instances from underrepresented classes, often requiring specialized techniques such as resampling methods, cost-sensitive learning, or ensemble methods tailored to handle imbalanced data distributions.

Handling imbalanced datasets involves employing various techniques to mitigate the impact of class imbalance on machine learning models. Here are some commonly used approaches:

Undersampling: Reduce the number of instances in the majority class(es) to match the minority class(es). This approach can help balance the dataset but may lead to loss of information.

Oversampling: Increase the number of instances in the minority class(es) by replicating or generating new instances. Techniques like SMOTE (Synthetic Minority Over-sampling Technique) create synthetic examples based on existing minority class instances.

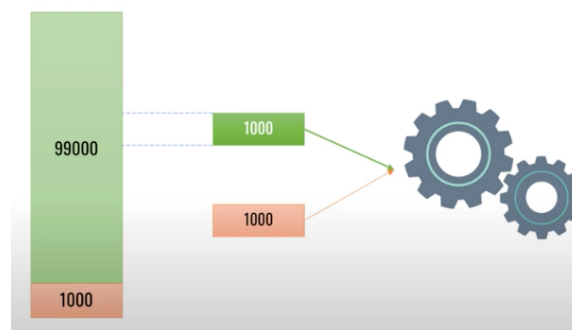


Figure 01: Undersampling majority class

Undersampling Majority Class:

Undersampling is a technique used to address class imbalance in datasets by reducing the number of instances in the overrepresented class(es) to match the number of instances in the underrepresented class(es).

For example, let's consider a scenario in credit card fraud detection where the dataset contains 100,000 instances, out of which only 1000 are instances of fraud (positive class), while the remaining 99000 instances are non-fraudulent (negative class). This dataset is heavily imbalanced, making it challenging for a machine learning model to accurately detect fraud.

To apply undersampling, one might randomly select 1000 instances from the negative class (non-fraudulent transactions) to match the number of instances in the positive class (fraudulent transactions), resulting in a balanced dataset with 1000 instances of each class. By doing so, the imbalance issue is mitigated, allowing the model to learn from a more representative distribution of data.

Drawback:

However, undersampling also comes with potential drawbacks. Since it reduces the amount of data available for training, it may lead to loss of valuable information and could potentially overlook important patterns present in the majority class. Therefore, it's essential to carefully consider the trade-offs and experiment with different undersampling ratios to find the optimal balance between addressing class imbalance and preserving information richness in the dataset.

Oversampling the Minority Class:

Oversampling techniques are used to increase the number of instances in the minority class to balance the dataset. In this scenario, we'll again consider the example of credit card fraud detection with 100,000 transactions, of which 1,000 are fraudulent and the rest are non-fraudulent. To balance the dataset using oversampling via duplicating data, we'll replicate instances of the minority class (fraudulent transactions) until it matches the size of the majority class (non-fraudulent transactions).

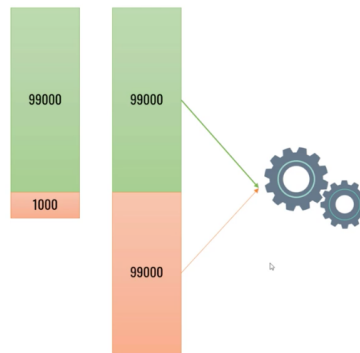


Figure 02: Oversampling minority class

Here's how it works:

1. Identify the minority class: In this case, the minority class is the fraudulent transactions.
2. Determine the oversampling ratio: This refers to the factor by which we need to increase the minority class size to match the majority class size. Since we have 99,000 instances of non-fraudulent transactions and only 1,000 instances of fraudulent transactions, we need to duplicate the fraudulent transactions 99 times.
3. Duplicate the data: Simply replicate each instance of the minority class (fraudulent transactions) to match the number of instances in the majority class (non-fraudulent transactions). This results in a new dataset where both classes have an equal number of instances.

Drawback:

While duplicating data is a straightforward approach, it can lead to the model being overly sensitive to the duplicated instances, potentially resulting in overfitting. Therefore, it's important to carefully consider the implications and explore other oversampling techniques, such as synthetic data generation methods like SMOTE, to mitigate these issues.

Here are some main techniques for oversampling:

Random Oversampling: This technique involves randomly duplicating instances from the minority class until it reaches the desired proportion relative to the majority class. While simple to implement, random oversampling may lead to **overfitting** due to duplicating the same instances.

SMOTE (Synthetic Minority Over-sampling Technique): SMOTE generates synthetic instances for the minority class by interpolating between existing minority class instances. It selects a minority class instance and its k nearest neighbors, then creates new instances along the

line segments connecting these instances. This helps in generating more diverse synthetic data compared to random oversampling.

Ensemble Method:

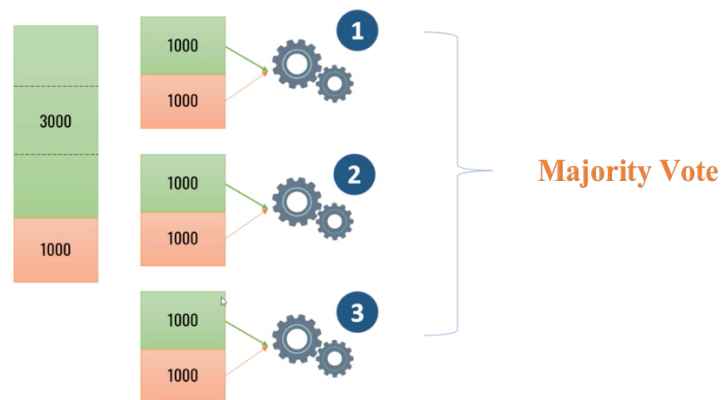


Figure 03: Ensemble Method (Using Majority Vote)

Example: Undersampling Majority Class

```
import pandas as pd
from imblearn.under_sampling import RandomUnderSampler

# Load the dataset
df = pd.read_csv('creditcard.csv')

# Separate features (X) and target variable (y)
X = df.drop('Class', axis=1)
y = df['Class']

# Instantiate the random undersampler
undersampler = RandomUnderSampler(random_state=42)

# Undersample the dataset
X_resampled, y_resampled = undersampler.fit_resample(X, y)

# Combine resampled features and target variable into a new DataFrame
df_resampled = pd.concat([pd.DataFrame(X_resampled, columns=X.columns),
pd.DataFrame(y_resampled, columns=['Class'])], axis=1)

# Save the balanced dataset to a new CSV file
df_resampled.to_csv('balanced_creditcard.csv', index=False)

print("Balanced dataset saved successfully!")
```

Example: Random Oversampling Minority Class

```
import pandas as pd
from imblearn.over_sampling import RandomOverSampler

# Load the dataset
df = pd.read_csv('creditcard.csv')

# Separate features (X) and target variable (y)
X = df.drop('Class', axis=1)
y = df['Class']

# Instantiate the random oversampler
oversampler = RandomOverSampler(random_state=42)

# Oversample the dataset
X_resampled, y_resampled = oversampler.fit_resample(X, y)

# Combine resampled features and target variable into a new DataFrame
df_resampled = pd.concat([pd.DataFrame(X_resampled, columns=X.columns),
pd.DataFrame(y_resampled, columns=['Class'])], axis=1)

# Save the balanced dataset to a new CSV file
df_resampled.to_csv('balanced_creditcard.csv', index=False)

print("Balanced dataset saved successfully!")
```